Lab # 12

Ejercicio 1:

a) Reducción de -B NOT

True = λx.λy.x False = λx.λy.y

Ifthen = λb.λx.λy.bxy

[NOT] = λb.ifthen b false True

NOT = λb.((λc.λx.λy.cxy) b false True)

→ β λb.((λx.λy.bxy) False True)

→ β λb.((λy.b False y) True

→ β λb.b False True ⟶ NOT = λb.b False True //

Recursión y los ciclos:

La recursión básicamente es cuando una función se llama a sí misma para repetir una operación hasta cumplir una condición para finalizar. En Lambda se usa "Y" para permitir comportamientos:

Y = λf.(λx.f(xx))(λx.f(xx)) y un ciclo infinito es cuando una expresión se repite pero no llega al final o un resultado concreto, por ejemplo: Ω = (λx.xx)(λx.xx).

Cuando es prudente y cuando no.

La Programación funcional es prudente cuando necesitamos trabajar con funciones reutilizables y realizar operaciones sin modificar datos originales, por ejemplo para calcular el cuadrado de varios numeros se puede usar $f = dx \cdot x^2$.

No es tan conveniente cuando ocupamos realizar muchos cambios constantes en la información ya que esto puede ser o resultar mas complicado por ejemplo en algo de inventarios es mejor usar programacion imperativa por los cambios en el stock.